

重定向

fopen/ open

- fopen 返回的是FILE *
- open返回的是 int
- fopen在Linux的底层实现就是open

- // open直接拿回来的文件描述符是3。
使用fopen打开一个文件，然后直接使用3这个文件描述符，开始写数据（write）。
数据最终观察到去文件里面。 ===》fopen底层就是open
- 文件描述符从3开始。
- fopen的底层是，open。 验证策略需要理解一下思路。

标准输入，输出，标准错误

- 这是三个标准流，在程序启动的时候。系统会默认开三个，
stdin,stdout,stderr
它们分别对应的文件描述符，是 0 1 2
- 要知道，标准输出的底层，也对应的有一个文件描述符
- 文件流的底层，其实也是使用的open 这种函数，拿到了一个文件描述符

- STDIN_FILENO
- STDOUT_FILENO
- STDERR_FILENO
- fileno()
宏定义拿

缓冲区的问题

缓冲区的问题。 文件的缓冲区：满缓冲区
stdout的缓冲区： 行缓冲区
stderr的缓冲区： 无缓冲

fopen也是一模一样的

- fopen的底层，其实就是使用的open
- 重定向这块的前提背景条件 。
 - printf就是死脑筋。往文件描述符为1的这个里面怼。
 - close()的时候。 到底做了什么事情。
close(1)。只是关文件描述符到文件对象之间的映射关系。
系统在初始创建的三个文件对象， 到最后的时候才会关。
 - 系统怎么分配文件描述符。从小到大，如果这个文件描述符没人用，就直接再次利用。

dup函数

- 就是将这个文件描述符复制一份， 新生成一个文件描述符。
这两个文件描述符指向同一个文件对象
- 生成的文件描述符， 是找的最小可用的

dup2函数

- 可以指定新的文件描述符。
- 新的文件描述符有两种情况：
没其他的在用，就分给你
其他的再用，就把之前的对应关系关掉，然后新的文件描述符和给定的文件描述符， 指向同一个文件对象